Gaussian Copula

General Principles

To model the dependence structure between random variables separately from their individual marginal distributions, we can use a **Gaussian Copula**. This approach assumes that the underlying latent dependency between variables follows a multivariate normal distribution.

The Gaussian copula is a foundational and widely used method. It has prominent applications across diverse domains, specifically in **quantitative finance** (e.g., modeling credit risk, CDOs, and implied correlations), **risk management**, and increasingly in **machine learning** (such as high-dimensional semi-parametric graphical models).

The model's goal is to construct a joint multivariate distribution where:

1. **The Marginal Distributions** can be any arbitrary distribution family (e.g., one Beta and one Poisson).
2. **The Dependence Structure** is dictated by a specified correlation matrix from a latent Gaussian space.

Copulas map the latent Gaussian variables into standard uniforms via the Normal Cumulative Distribution Function (CDF), and then project those uniforms onto the desired target distributions using their respective Inverse CDFs (Quantile functions).

Because the copula links the variables via non-linear rank-based transformations, the linear Pearson correlation on the final variables is not preserved. However, rank correlation metrics like **Kendall's** τ and **Spearman's** ρ_s persist and can be mapped back to the origin Gaussian correlation mathematically.

Considerations

> **Note**
>
> When applying the Gaussian copula, keep the following facts in mind:
>
> 1. **Dependence Range**: The correlation parameter ρ ranges from -1 to 1, allowing the Gaussian copula C_ρ to interpolate between complete dependence and independence.
> 2. **Concordance Measures**: The Gaussian copula has simple closed-form formulas for concordance measures:
>   - Kendall's τ: $\tau_\rho = \frac{2}{\pi}\arcsin(\rho)$
>   - Spearman's ρ_S: $\rho_S = \frac{6}{\pi}\arcsin\left(\frac{\rho}{2}\right)$
> 3. **Symmetries**:
>   - *Radial Symmetry*: The copula is equal to its own survival copula. Points are symmetrically scattered around the counterdiagonal. This fails to capture asymmetries such as joint negative asset returns or early failures in survival models.
>   - *Exchangeability*: The bivariate copula is invariant under permutations of its arguments. Combined with inhomogeneous marginals, it can lead to unintuitive modeling of joint events.
> 4. **Tail Independence**: The Gaussian copula does not exhibit tail dependence for $\rho \in (-1, 1)$. This means it may not be suitable for modeling extreme co-movements in the tails of the distribution, as it underestimates the probability of joint extreme events. This limitation is highly relevant when assessing extreme risks in finance or weather.

Example

Below we demonstrate how to implement a Gaussian Copula in Python, R, and Julia, modeling a bivariate relationship. For Python, we use the BI framework to define custom marginal distributions and perform Bayesian inference.

Python

```
import jax.numpy as jnp
from jax import random
from jax.scipy.stats import norm
```

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
import numpyro.distributions as dist
from numpyro.distributions import constraints
from tensorflow_probability.substrates import jax as tfp
import jax.scipy.special as jss
from BI import bi

# Core parameters
n = 1000
spearman_rho = 0.6
pearson_rho = 2 * np.sin((np.pi / 6) * spearman_rho)

# Covariance matrix for bivariate standard normal
Sigma = jnp.array([
    [1.0**2, pearson_rho * 1.0 * 1.0],
    [pearson_rho * 1.0 * 1.0, 1.0**2]
])
mu = jnp.array([0.0, 0.0])
Sigma_chol = jnp.linalg.cholesky(Sigma)

seed = 123
rng_key = random.PRNGKey(seed)

# Marginal parameters (Beta and Poisson)
alpha_param, beta_param = 2.0, 5.0
lambda_param = 3.0

# ------------------------------------------------------------------
# 1. Raw Functional Approach
# ------------------------------------------------------------------
samples_normal = random.multivariate_normal(rng_key, mean=mu, cov=Sigma, shape=(n,))
samples_uniform = norm.cdf(samples_normal)

u1 = np.asarray(samples_uniform[:, 0])
u2 = np.asarray(samples_uniform[:, 1])

x_beta = stats.beta.ppf(u1, a=alpha_param, b=beta_param)
x_poisson = stats.poisson.ppf(u2, mu=lambda_param)
```

```
# ------------------------------------------------------------------
# 2. Built-in Copula (Beta & Poisson) with BI
# ------------------------------------------------------------------
m = bi('cpu')

class BetaPoissonMarginal(dist.Distribution):
    """
    A custom marginal distribution wrapper combining Beta and Poisson distributions.

    This class bypasses NumPyro's constraint that multivariate copula marginals
    must belong to the exact same distribution family. By grouping both variables into
    a single vector space of size 2, it allows the Gaussian copula to map the first
    latent dimension to a Beta distribution and the second dimension to a Poisson distributio
    """
    # The support is a continuous unbounded vector during HMC sampling for the latent space
    support = constraints.real_vector

    def __init__(self, alpha, beta, lam, validate_args=None):
        """
        Initializes the custom marginal joint distribution.

        Args:
            alpha: Shape parameter (alpha) for the first variable (Beta).
            beta: Shape parameter (beta) for the first variable (Beta).
            lam: Rate parameter (lambda) for the second variable (Poisson).
            validate_args: Verification flag for NumPyro's distribution base class.
        """
        self.alpha = alpha
        self.beta = beta
        self.lam = lam
        # batch_shape=(2,) tells NumPyro that this distribution evaluates two independent
        # variables simultaneously per sample. event_shape=() means each evaluates to a scala
        super().__init__(batch_shape=(2,), event_shape=(), validate_args=validate_args)

    def log_prob(self, value):
        """
        Computes the log probability density (or mass) for the observations.

        This is called during inference (HMC/NUTS) to compute the likelihood of the model.
        """
        # value[..., 0] contains the observations/samples for the Beta variable
        v1 = value[..., 0]
```

```
        # value[..., 1] contains the observations/samples for the Poisson variable
        v2 = value[..., 1]

        # Calculate individual log probabilities using their respective NumPyro distributions
        lp1 = dist.Beta(self.alpha, self.beta).log_prob(v1)
        lp2 = dist.Poisson(self.lam).log_prob(v2)

        # Stack the log probabilities back into the shape expected by the copula (..., 2)
        return jnp.stack([lp1, lp2], axis=-1)

    def cdf(self, value):
        """
        Computes the Cumulative Distribution Function (CDF) for mapping.

        This function transforms the observed data back into uniform probabilities [0, 1]
        during inference, allowing the copula to link them to the latent Gaussian space.
        """
        v1 = value[..., 0]
        v2 = value[..., 1]

        # Use TensorFlow Probability's Regularized Incomplete Beta function (betainc)
        # because it supports JAX autodiff (gradients) with respect to alpha and beta paramet
        cdf1 = tfp.math.betainc(self.alpha, self.beta, v1)

        # JAX's regularized upper incomplete gamma function (gammaincc) is used as an
        # alternative to compute the Poisson CDF. Floor is used to step-function continuous
        # P(X <= k) = Q(k+1, lambda) = gammaincc(k+1, lambda)
        cdf2 = jss.gammaincc(jnp.floor(v2) + 1.0, self.lam)

        return jnp.stack([cdf1, cdf2], axis=-1)

    def icdf(self, u):
        """
        Computes the Inverse Cumulative Distribution Function (Quantile function).

        This is used for forward sampling (generating new data from the model).
        It transforms standard uniform values u ~ U[0, 1] generated by the copula
        into the original target scales (Beta and Poisson).
        """
        import numpy as np
        import scipy.stats as stats
```

```
        # Scipy operates on CPU NumPy arrays, so we must coerce JAX arrays safely
        u1 = np.asarray(u[..., 0])
        u2 = np.asarray(u[..., 1])

        # Apply the Scipy Point Percentile Function (ppf/icdf) for Beta and Poisson
        x_b = stats.beta.ppf(u1, a=np.asarray(self.alpha), b=np.asarray(self.beta))
        x_p = stats.poisson.ppf(u2, mu=np.asarray(self.lam))

        # Coerce back to JAX arrays to integrate into the regular BI/JAX pipeline properly
        return jnp.stack([jnp.array(x_b), jnp.array(x_p)], axis=-1)

samples_bi = m.dist.gaussian_copula(
    marginal_dist=BetaPoissonMarginal(alpha_param, beta_param, lambda_param),
    correlation_cholesky=Sigma_chol,
    sample=True,
    shape=(n,),
    seed=seed
)
x_b1 = samples_bi[:, 0]
x_b2 = samples_bi[:, 1]

# ----------------------------------------------------------------------
# 3. Inference model with BI
# ----------------------------------------------------------------------
def model(x_b1, x_b2):
    alpha_est = m.dist.exponential(0.1, name='alpha_est')
    beta_est = m.dist.exponential(0.1, name='beta_est')
    lambda_est = m.dist.exponential(0.1, name='lambda_est')

    rho_est = m.dist.lkj_cholesky(2, 2.0, name='rho_est')

    obs_data = jnp.stack([x_b1, x_b2], axis=-1)
    m.dist.gaussian_copula(
        marginal_dist=BetaPoissonMarginal(alpha_est, beta_est, lambda_est),
        correlation_cholesky=rho_est,
        obs=obs_data,
        name='obs'
    )

m.data_on_model = {'x_b1': x_b1, 'x_b2': x_b2}
m.fit(model, num_samples=300, num_warmup=100, num_chains=1, progress_bar=True)
m.sampler.print_summary()
```

R

```r
library(tidyverse)
library(patchwork)

n_obs <- 1000
rho <- 0.6
lambda1 <- 2
lambda2 <- 4

# Covariance matrix
sigma <- matrix(c(1, rho, rho, 1), nrow = 2)
L <- chol(sigma)

# Step 1: Sample from bivariate standard normal
set.seed(1)
Z <- matrix(rnorm(n = n_obs * 2), nrow = 2)
Z <- t(L %*% Z)

# Step 2: Transform the samples into standard uniforms using the CDF of the standard normal
u <- pnorm(Z)

# Step 3: Transform into exponential marginals using quantile functions
y1 <- qexp(u[, 1], rate = lambda1)
y2 <- qexp(u[, 2], rate = lambda2)
```

Julia

```julia
using Distributions
using LinearAlgebra
using Random

n_obs = 1000
rho = 0.6
lambda1 = 2.0
lambda2 = 4.0

# Covariance matrix
Σ = [1.0 rho; rho 1.0]

# Step 1: Sample from a bivariate standard normal
```

```
dist_N = MvNormal(zeros(2), Σ)
Random.seed!(1)
Z = rand(dist_N, n_obs)'

# Step 2: Transform the samples into standard uniforms using the CDF of the standard normal
u = cdf.(Normal(), Z)

# Step 3: Transform into exponential marginals using quantile functions
target1 = Exponential(1.0 / lambda1)
target2 = Exponential(1.0 / lambda2)

y1 = quantile.(target1, u[:, 1])
y2 = quantile.(target2, u[:, 2])
```

Mathematical Details

To model the joint distribution analytically, we can combine the marginal distributions with the Gaussian copula. For two variables X_1 and X_2 with marginal distributions $f_1(X_1)$ and $f_2(X_2)$, the joint log-density is given by combining the multivariate normal dependence and the uniform transformations:

$$\log h(\mathbf{X}) = \log f_{\Sigma}(z_1, z_2|\Sigma) - \sum_{i=1}^{2} \log \phi(z_i) + \sum_{i=1}^{2} \log f_i(X_i)$$

Here, $z_i = \Phi^{-1}(F_i(X_i))$ relies on the cumulative distribution functions F_i for each marginal. f_{Σ} is the bivariate normal density with correlation matrix Σ, and $\phi(z_i)$ represents independent standard normal densities.

Notes

> **Note**
>
> Step to use the gaussian copula: 1) sample from a bivariate standard normal (mean = 0 and sigma = 1) specifying the pearson correlation parameter 2) transform the samples into standard uniforms using the CDF of the standard normal, pnorm() in R 3) transformed the standard uniform samples into whichever distributions you want using the quantile functions of the target distributions 4) the resulting transformed variables will have the desired shape and have dependence structure, but the pearson coefficient is not preserved 5) the spearman coefficient is preserved

Reference(s)

Books & Comprehensive References

1. ***An Introduction to Copulas* (Roger B. Nelsen)** — A classic textbook covering copula theory (not specific to Gaussian only, but excellent foundation). (Springer)
2. ***Dependence Modeling with Copulas* (Harry Joe)** — Advanced reference focusing on copula models and estimation methods. (copula.stat.ubc.ca)
3. ***Copulae in Mathematical and Quantitative Finance*** — A workshop proceedings book with papers on copulas. (Springer)
4. ***Credit Models and the Crisis* (Damiano Brigo et al.)** — A book chapter that walks through Gaussian copula models in credit risk and implied correlation. (O'Reilly)
5. ***Financial Engineering with Copulas Explained* (Mai & Scherer)** — Focused on applications of copula methods in finance. (Springer)

Online Tutorials & Articles

6. **A Gentle Introduction: The Gaussian Copula** — Detailed breakdown of Gaussian copula properties and Stan implementation. (bggj.is)
7. **TensorFlow Probability Copulas Primer** — A gentle introduction with code examples for Gaussian copulas. (TensorFlow)
8. **Simulation with Copulas** — A practical introduction including formula for Gaussian copula and simulation context. (Langtian Ma)
9. **Gaussian Copula Model summary** — Overview and discussion of implementation in credit portfolios. (ResearchGate)
10. **Copula Resources** — A curated list linking lecture notes, examples, and software. (Davis Vaughan)

Implementations & Code Examples

11. **R Package: GaussCop** — Tools for density evaluation and sampling with arbitrary margins. (Martin Lysy)
12. **Bivariate Gaussian Copula Simulation** — Example code for simulation. (RPubs)
13. **MATLAB `copulapdf` documentation** — Built-in MATLAB function for Gaussian copula density. (MathWorks)

Advanced & Research Papers

14. **Semi-parametric Gaussian Copula Models in ML (Thesis)** — Deep dive into Gaussian copulas in machine learning contexts. (Université de Bâle Édition Numérique)

15. **Empirical copula processes by Gaussian processes** — More theoretical perspective on empirical copula processes. (arXiv)
16. **High-Dimensional Gaussian Copula Graphical Models** — Research on semiparametric graphical modeling with Gaussian copulas. (arXiv)

Quick Practical Notes

- Gaussian copulas are widely used for dependence modeling but **don't capture tail dependence well**, which matters in finance and extremes. (Medium)
- When starting out, the **TensorFlow primer** or simulations in R/Python can make abstract concepts concrete.
- For a solid theoretical grounding before implementation, begin with Nelsen's book, then expand to Joe's dependence modeling text.